THIẾT KẾ HỆ THỐNG NHÚNG GIÁM SÁT NHIỆT ĐỘ - ĐỘ ẨM VÀ ĐIỀU KHIỂN QUẠT THÔNG GIÓ TỰ ĐỘNG TRONG KHÔNG GIAN KÍN

HỌ VÀ TÊN	MSSV
Nguyễn Minh Chiến	24139006
Lê Trương Long	24139028
Phan Hà Hoàng Nhân	24139033
Nguyễn Quang Minh	24139030

MỤC ĐÍCH CỦA HỆ THỐNG

1. Thu thập và giám sát môi trường liên tục:

- Đo đạc chính xác nhiệt độ và độ ẩm trong không gian kín bằng cảm biến theo thời gian thực.
- Hiển thị trực quan các thông số và trạng thái hoạt động lên màn hình LCD tại chỗ và đồng bộ lên web.

2. Điều khiển làm mát tự động thông minh (Automatic):

- Hệ thống tự tính toán mức nhiệt/ẩm để băm xung PWM điều chỉnh tốc độ quạt tương ứng thay vì chỉ bật/tắt đơn thuần, giúp duy trì môi trường ổn định và tối ưu hóa điện năng tiêu thụ.

3. Điều khiển thiết bị đa chế độ:

- Tại chỗ - vận hành: Vặn biến trở để chỉnh trực tiếp công suất quạt theo ý muốn ngay trên mạch.
- Từ xa: Giám sát, nhận thông báo và điều khiển bật/tắt/chỉnh tốc độ quạt thông qua Web điện thoại.

4. Đảm bảo an toàn và đáp ứng thời gian thực (Real-time & Safety):

- Phản ứng khẩn cấp: Khi nhiệt độ tăng vọt bất thường, hệ thống lập tức ép quạt chạy 100% công suất và bắn cảnh báo tức thì về Web.
- Tự phục hồi: Tự động khởi động lại khi phần mềm bị treo và tự khôi phục lại cài đặt cũ sau khi gặp sự cố mất điện đột ngột.

CƠ SỞ LÝ THUYẾT

1. Hệ điều hành thời gian thực (FreeRTOS trên ESP32)

- Lập lịch ưu tiên (Preemption): Luôn ưu tiên thực thi tác vụ khẩn cấp (xử lý khi nhiệt độ quá nhiệt) trước các tác vụ phụ (cập nhật LCD, gửi log).
- Đồng bộ IPC: Dùng Queue truyền dữ liệu đo đạc an toàn giữa các luồng; dùng Mutex chống tranh chấp bus I2C giữa các thiết bị ngoại vi.

2. Kỹ thuật PWM & Điều khiển quạt 4 dây

- Điều chế xung (PWM): Thay đổi chu kỳ nhiệm vụ ($Duty\ Cycle = T_ON/T \times 100\%$) với tần số chuẩn 25 kHz để chỉnh mượt tốc độ quạt từ 0% đến 100%, tiết kiệm điện năng.
- Đọc tốc độ (Dây TACH): Đếm số xung Hall gửi về qua ngắt GPIO để tính tốc độ vòng quay thực tế (RPM):

$$"RPM" = "Số\ xung/giây"\ /2 \times 60$$

3. Bộ chuyển đổi ADC & Điều khiển tại chỗ

- Đọc biến trở qua ADC: Chuyển đổi điện áp xoay tay ($0 - 3.3"V"$) thành giá trị số 12-bit ($0 - 4095$), sau đó ánh xạ (*map*) thẳng sang dải xung PWM ($0\% - 100\%$).
- Lọc vùng chết (Deadband): Sử dụng deadband để lọc nhiễu điện áp ADC, giúp duy trì tốc độ quạt ổn định, tránh dao động liên tục khi ở Operator mode.

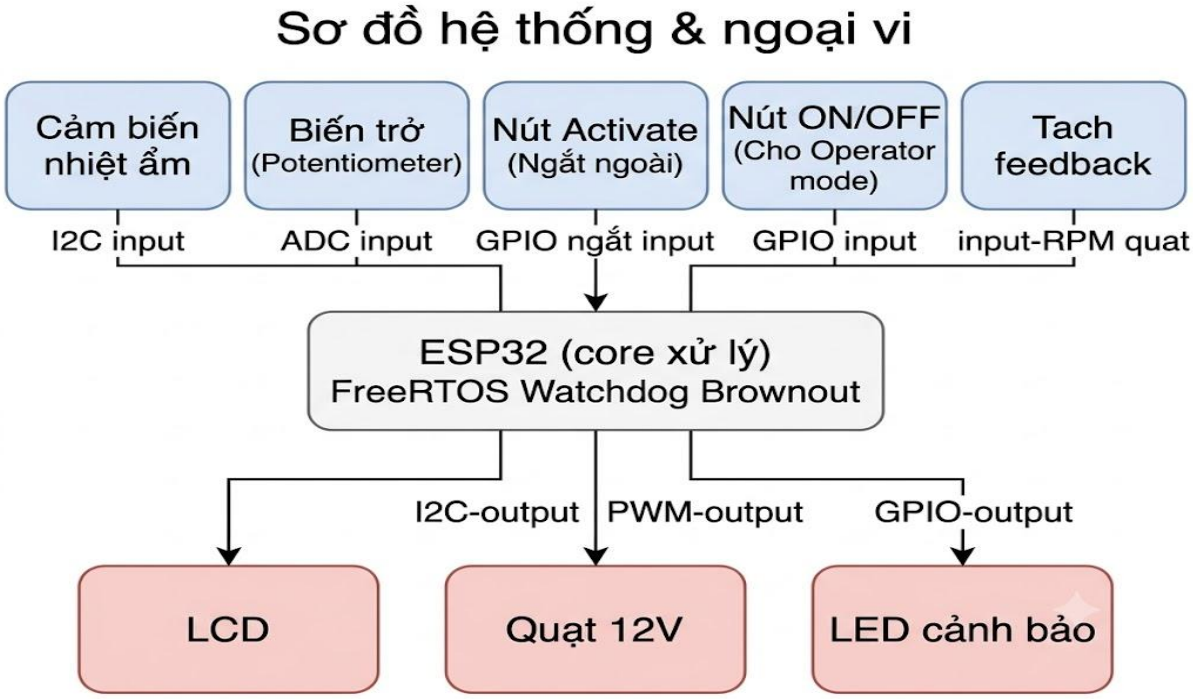
4. Thuật toán điều khiển & An toàn hệ thống

- Chế độ Auto: Tự động điều chỉnh cấp tốc độ quạt theo nhiệt độ kết hợp dải trễ (*Hysteresis*) để tránh quạt bật/tắt chập chờn.
- Chế độ User Control & Khẩn cấp: Người dùng can thiệp qua biến trở hoặc Web; quạt lập tức ép chạy 100% công suất khi phát hiện quá nhiệt nguy hiểm.
- Tự khôi phục: Dùng Watchdog Timer (WDT) tự Reset khi vi điều khiển bị treo và lưu cấu hình vào bộ nhớ Flash/NVS để tự phục hồi trạng thái sau khi mất điện.

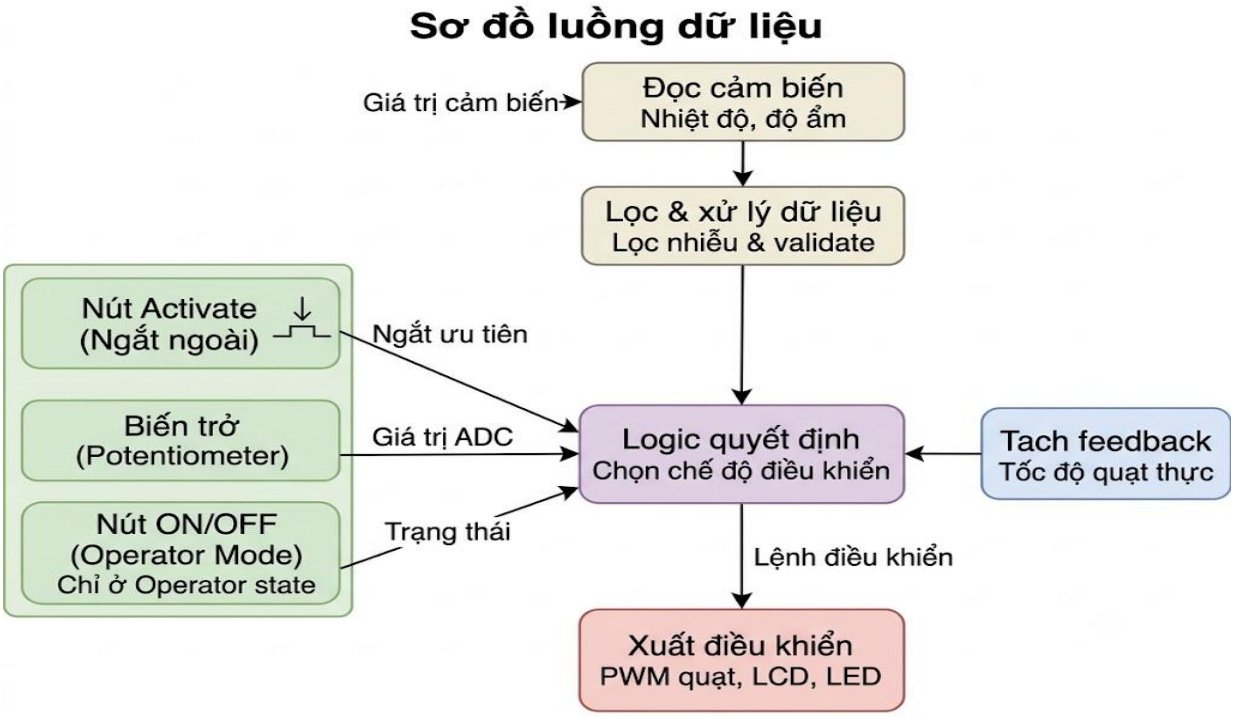
NGOẠI VI

Ngoại vi	Loại	Mục đích
Cảm biến nhiệt độ, độ ẩm (I2C)	Input	Đọc nhiệt độ, độ ẩm từ môi trường
LCD (I2C)	Output	Hiển thị thông tin, trạng thái môi trường
Nút Activate (GPIO + External Interrupt)	Input	Kích hoạt chuyển sang Operator mode
Biến trở (ADC1)	Input	Đọc giá trị điện áp để quy đổi sang công suất quạt
Nút ON/OFF (GPIO input)	Input	Bật/tắt quạt trong Operator mode
Quạt 12V (PWM peripheral)	Output	Điều khiển tốc độ quạt
Phản hồi từ quạt (Tach feedback)	Input	Đọc phản hồi RWM quạt thực tế
LED cảnh báo (GPIO output)	Output	Phát tín cảnh báo độc lập với app
Watchdog Timer	Reliability	Tự reset hệ thống nếu bị treo

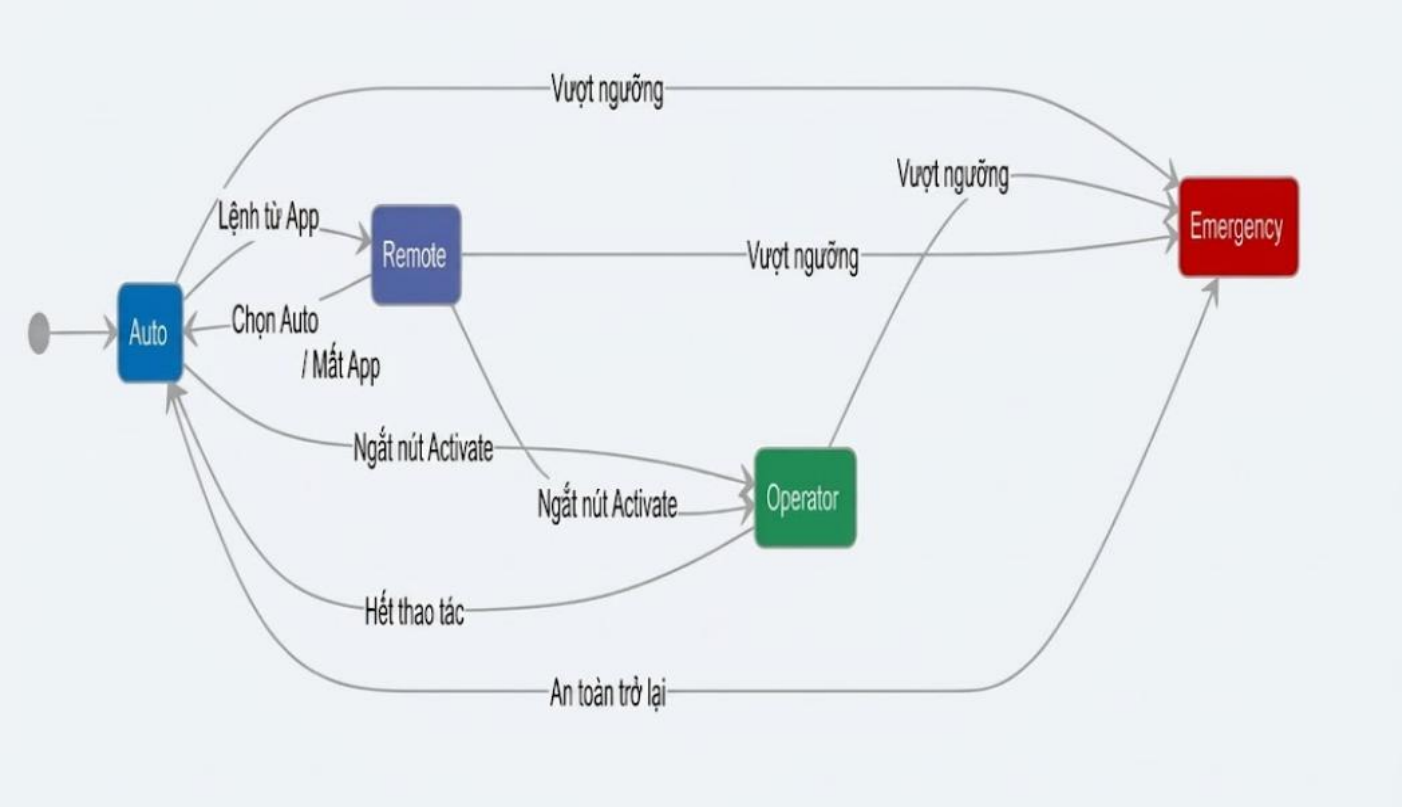
Sơ đồ hệ thống & ngoại vi:



Sơ đồ luồng dữ liệu:



Sơ đồ trạng thái:



Bảng chuyển trạng thái:

Từ	Sự kiện	Đến	Hành động
Auto	Lệnh từ App	Remote	Nhận setpoint từ app
Remote	Chọn Auto / mất kết nối App	Auto	Trả quyền cho thuật toán hệ thống
Auto, Remote	Nhấn Activate	Operator	Bắt đầu đọc ADC, ON/OFF có hiệu lực
Operator	Timeout	Auto	Khoá biến trở và ON/OFF
Auto, Remote, Operator	T vượt ngưỡng trên	Emergency	Quạt đạt 100%, LED cảnh báo
Emergency	T ≤ ngưỡng dưới	Auto	Trả về điều khiển tự động

PHÂN TÍCH CẤU TRÚC PHẦN CỨNG VÀ THANH GHI NGOẠI VI

Hệ thống vi điều khiển ESP32 quản lý toàn bộ ngoại vi thông qua cơ chế Memory-Mapped I/O (MMIO). Các thanh ghi điều khiển có độ rộng tiêu chuẩn 32-bit và được truy xuất qua bus ngoại vi APB.

1.1. Tổng quan không gian bộ nhớ ngoại vi

Ngoại vi điều khiển	Ký hiệu Base Address	Địa chỉ vật lý	Chức năng trong hệ thống
LEDC	DR_REG_LEDC_BASE	0x3FF59000	Xuất xung PWM 25kHz điều khiển tốc độ quạt
I2C 0	DR_REG_I2C_EXT_BASE	0x3FF53000	Bus I2C giao tiếp cảm biến AHT20 và màn hình LCD
GPIO MUX	DR_REG_GPIO_BASE	0x3FF44000	Quản lý logic I/O, ngắt ngoài, tín hiệu TACH và LED
ADC1	DR_REG_SENS_BASE	0x3FF48800	Bộ chuyển đổi SAR ADC đo điện áp từ biến trở

1.2. Cấu hình thanh ghi điều chế độ rộng xung (LEDC)

Thanh ghi/nhóm		**Độ rộng**	**Chức năng**
LEDC_HSTIMER0_CONF_REG (0x3FF59140)		[31:0]	Chọn nguồn cấp xung clock, thiết lập bộ chia tần số, quy định độ phân giải.
Bit fields	LEDC_TICK_SEL_HSTIMER0	[25]	Lựa chọn nguồn để cấp xung clock đầu vào: 1: APB_CLK 0: REF_TICK
	LEDC_CLK_DIV_HSTIMER0	[22:5]	Chia tần số clock gốc 80 MHz xuống nhịp đếm phù hợp.
	LEDC_HSTIMER0_DUTY_RES	[4:0]	Quy định chu kỳ đếm từ 0 - 1023 để phân giải độ rộng xung PWM quạt thành 1024 nấc tốc độ.
LEDC_HSCH0_CONF0_REG (0x3FF59000)		[31:0]	Chọn timer, enable output và mức idle.
Bit fields	LEDC_TIMER_SEL_HSCH0	[1:0]	Dùng để chọn Timer nguồn (Timer 0) cấp xung cho kênh High-Speed Channel 0.
	LEDC_SIG_OUT_EN_HSCH0	[2]	Dùng để cho phép xuất xung PWM ra chân GPIO.

	LEDC_IDLE_LV_HSCH0	[3]	Dùng để đặt mức 0 (LOW) khi quạt không chạy.
LEDC_HSCH0_HPOINT_REG (0x3FF59004)		[31:0]	Xác định mốc đếm để tín hiệu kênh 0 bắt đầu bật lên mức cao.
Bit fields	LEDC_HPOINT_HSCH	[19:0]	Đặt giá trị mốc đếm mà tại đó tín hiệu PWM xuất ra chân quạt sẽ bật lên mức HIGH (1).
LEDC_HSCH0_DUTY_REG (0x3FF59008)		[31:0]	Xác định thời gian tín hiệu giữ mức cao trước khi hạ về mức thấp
Bit fields	LEDC_DUTY_HSCH0	[24:4]	Lưu giá trị Duty Cycle số nguyên. Khi bộ đếm Timer 0 đếm từ 0 và đạt tới giá trị này, tín hiệu PWM trên chân GPIO sẽ bị hạ xuống mức LOW.
	LEDC_DUTY_HSCH0	[3:0]	Dùng để làm mịn xung (chia nhỏ mỗi nấc đếm thành $\frac{1}{16}$ nấc). Khi không dùng tính năng dither, 4 bit này mặc định bằng 0
LEDC_HSCH0_CONF1_REG (0x3FF5900C)		[31:0]	Áp dụng giá trị Duty mới ra chân quạt và điều khiển tự động tăng/giảm tốc độ (fading) cho kênh 0.

Bit fields	LEDC_DUTY_SCALE_HSCH0	[9:0]	Quy định biên độ thay đổi của Duty ở mỗi bước.
	LEDC_DUTY_CYCLE_HSCH0	[19:10]	Quy định độ trễ (tính bằng số chu kỳ sóng PWM) giữa mỗi lần nhích/thay đổi giá trị Duty.
	LEDC_DUTY_NUM_HSCH0	[29:20]	Quy định tổng số bước (số lần) thực hiện tăng hoặc giảm Duty Cycle trước khi dừng quá trình Fade.
	LEDC_DUTY_START_HSCH0	[31]	Quy định hướng biến đổi của Duty Cycle: 1: Tăng dần Duty (Fade Up) 0: Giảm dần Duty (Fade Down)

- Để cấu hình độ phân giải của Duty Cycle. Tại tần số 25kHz với xung nhịp nguồn APB Clock (80MHz), độ phân giải tối đa được tính bằng công thức:

$$\text{Resolution}_{\max} = \log_2 * \left(\frac{80 \times 10^6}{25000} \dashv \right) \approx 11.64 \text{ bit}$$

Nhóm chọn thiết lập giá trị 10-bit (0x0A), cung cấp dải điều khiển từ 0 đến 1023.

1.3. Cấu hình thanh ghi giao tiếp I2C Master

Thanh ghi/nhóm	Độ rộng	Chức năng

I2C_CTR_REG (0x0004)		[31:0]	Cấu hình chế độ hoạt động chính và bật/tắt luồng truyền nhận bus I2C.
Bit fields	I2C_RX_LSB_FIRST	[7]	Chọn thứ tự truyền/nhận các bit trong mỗi byte dữ liệu. 0: Truyền/nhận MSB-first 1: Truyền/nhận LSB-first
	I2C_TX_LSB_FIRST	[6]	
	I2C_TRANS_START	[5]	Ghi 1 vào bit này để kích hoạt bắt đầu truyền lệnh/dữ liệu trên bus.
	I2C_MS_MODE	[4]	Chọn chế độ Master (1) hoặc Slave (0).
	I2C_SCL_FORCE_OUT	[1]	Chọn chế độ điều khiển cho các chân SDL. 0: Chế độ Open-Drain 1: Ép ngõ ra thành Direct Output
	I2C_SDA_FORCE_OUT	[0]	Chọn chế độ điều khiển cho các chân SCA. 0: Chế độ Open-Drain 1: Ép ngõ ra thành Direct Output
I2C_SR_REG (R0) (0x0008)		[31:0]	Thanh ghi dùng để giám sát trạng thái thời gian thực của bus I2C.
Bit fields	I2C_TXFIFO_CNT	[23:18]	Số byte dữ liệu đang nằm trong bộ đệm gửi (TX FIFO).
	I2C_RXFIFO_CNT	[13:8]	Số byte dữ liệu đang nằm trong bộ đệm nhận (RX FIFO) chờ CPU đọc.

	I2C_BUS_BUSY	[4]	Báo bus I2C đang bận truyền dữ liệu (1) hay rảnh (0).
	I2C_ARB_LOST	[3]	Khi bộ điều khiển I2C mất quyền kiểm soát đường SCL, trường bit này chuyển sang giá trị 1 – dừng phiên truyền dử liệu - để tránh xung đột dữ liệu.
	I2C_TIME_OUT	[2]	Tự động bật lên mức 1 khi thời gian nhận một bit dữ liệu vượt quá ngưỡng quy định (hết thời gian chờ - Timeout).
	I2C_ACK_REC	[0]	Phản hồi ACK nhận được từ cảm biến 0: Có ACK 1: NACK/Không thấy cảm biến
I2C_FIFO_CONF_REG (0x3FF53018)		[31:0]	Quản lý và cài đặt các ngưỡng nạp/rút dữ liệu cho bộ đệm FIFO.
Bit fields	I2C_NONFIFO_EN	[10]	Bật (1) chế độ truy cập APB non-FIFO, tắt (0) dùng bộ đệm FIFO thông thường.
	I2C_TXFIFO_EMPTY_THRHD	[9:5]	Đặt ngưỡng cảnh báo khi bộ đệm gửi (TX FIFO) cạn (hoặc dưới ngưỡng thiết lập).
	I2C_RXFIFO_FULL_THRHD	[4:0]	Đặt ngưỡng cảnh báo khi bộ đệm nhận (RX FIFO) đầy.
I2C_DATA_APB_REG (0x3FF5301C)		[31:0]	Là cổng đọc/ghi trực tiếp byte dữ liệu giữa CPU và bộ đệm I2C FIFO.

Bit fields	I2C_FIFO_RDATA	[7:0]	Chứa 1 byte dữ liệu vừa nhận được từ ngoại vi.
	I2C_FIFO_WDATA	[7:0]	Chứa 1 byte dữ liệu chuẩn bị phát ra bus I2C.
I2C_COMDn_REG (n =0,1..15) (0x3FF53058 + $n \times 4$)		[31:0]	Thanh ghi chứa lệnh thực thi của I2C Master:
Bit fields	I2C_COMMANDn_DONE	[31]	Cờ báo lệnh thứ n đã thực thi hoàn tất (chuyển lên mức 1 khi chạy xong)
	I2C_COMMANDn	[13:0]	Thiết lập nội dung thực thi của lệnh thứ n, gồm: 1. mã lệnh op_code (RSTART, WRITE, READ, STOP, END) 2. Số byte cần truyền/nhận (Byte_num) 3. Cấu hình kiểm tra bit phản hồi ACK/NACK
I2C_SCL_LOW_PERIOD (R/W) (0x3FF53000)		[31:0]	Quy định khoảng thời gian xung đồng hồ SCL ở mức LOW (0) và mức HIGH (1).
Bit fields	I2C_SCL_LOW_PERIOD	[13:0]	Số chu kỳ clock nội (APB_CLK 80 MHz) giữ SCL ở mức LOW.
I2C_SCL_HIGH_PERIOD (R/W) (0x3FF53038)		[9:5]	Đặt ngưỡng cảnh báo khi bộ đệm gửi (TX FIFO) cạn (hoặc dưới ngưỡng thiết lập).

Bit fields	I2C_SCL_HIGH_PERIOD	[13:0]	Số chu kỳ clock nội (APB_CLK 80 MHz) giữ SCL ở mức HIGH.

1.4. Cấu hình thanh ghi GPIO và Ngắt ngoài

Thanh ghi/nhóm		Độ rộng	Chức năng
GPIO_ENABLE_REG (0x3FF44020)		[31:0]	Cấu hình chiều Input/Output cho các chân GPIO. Mỗi chân tương ứng 1 bit. 1: chế độ Output 0: chế độ Input
GPIO_OUT_W1TS_REG (0x3FF44008)		[31:0]	Điều khiển điện áp đầu ra mà không gây xung đột Read-Modify-Write
GPIO_OUT_W1TC_REG (0x3FF4400C)		[31:0]	
GPIO_PINn_REG ((0x3FF44088 + n*4)		[31:0]	Cấu hình các đặc tính như ngắt, pull-up/down cho từng chân GPIO
Bit fields	GPIO_PINn_INT_TYPE	[9:7]	Thiết lập sự kiện ngắt ngoài: 0: tắt ngắt 1: cạnh lên 2: cạnh xuống 3: cả 2 cạnh

1.5. Cấu hình thanh ghi ADC1

Bộ chuyển đổi SAR ADC đo điện áp từ biến trở để tham chiếu tốc độ quạt.

Thanh ghi/nhóm		Độ rộng	Chức năng
SENS_SAR_ATTEN1_REG (0x3FF4883C)		[31:0]	Cấu hình hệ số suy hao điện áp đầu vào cho các chân ngoại vi thuộc bộ SAR ADC1
Bit fields	SAR_ATTEN_ADC1_CHn	2 bit / kênh	Thiết lập suy hao điện áp cho từng kênh tương ứng
SENS_SAR_READ_STATUS1_REG (0x3FF48858)		[31:0]	Lưu trữ trạng thái hoạt động hiện tại và dữ liệu kết quả chuyển đổi của bộ ADC1
Bit fields	SAR1_DONE	[16]	Cờ báo hiệu phần cứng đã hoàn tất quá trình lấy mẫu: 1: Dữ liệu đã sẵn sàng để đọc 0: Đang trong quá trình chuyển đổi
Bit fields	SAR1_DATA	[11:0]	Chứa kết quả chuyển đổi tương tự - số của tín hiệu biến trở. Dải giá trị trả về dạng số nguyên từ 0 đến 4095

PHÂN TÍCH CƠ CHẾ HOẠT ĐỘNG CỦA THƯ VIỆN

2.1. Phân tầng kiến trúc phần mềm

Hệ thống được thiết kế theo nguyên tắc đóng gói abstraction, với luồng thực thi đi qua 4 lớp:

1. Lớp Ứng dụng: Các Task FreeRTOS quản lý State Machine, thuật toán lọc Deadband, và hàm ISR.
2. Lớp Thư viện Component: Thư viện Wire.h (I2C), LiquidCrystal_I2C, và AHT20.
3. Lớp Trừu tượng phần cứng (ESP-IDF HAL): Các file esp32-hal-i2c.c, esp32-hal-ledc.c chuyển đổi tham số cấp cao thành giá trị thanh ghi.
4. Lớp Phần cứng vật lý: Các thanh ghi MMIO điều khiển trực tiếp điện áp và tín hiệu chân vi điều khiển.

2.2. Cơ chế can thiệp phần cứng của API LEDC

Trong phiên bản ESP32 Core v3.x, quy trình khởi tạo PWM được tối ưu qua hàm ledcAttach(pin, freq, resolution).

1. Định tuyến động (Dynamic Routing): Hàm tự động dò tìm Timer và Channel đang rảnh trong bộ nhớ quản lý nội bộ.
2. Cấu hình thanh ghi: Tính toán hệ số chia (Divisor) và ghi vào LEDC_HSTIMERx_CONF_REG.
3. Kết nối ma trận GPIO: Gọi hàm nội bộ esp_rom_gpio_connect_out_signal(). Cơ chế này can thiệp vào thanh ghi GPIO_FUNCx_OUT_SEL_CFG_REG, định tuyến tín hiệu PWM sinh ra từ khối LEDC bên trong chip ra đúng chân GPIO vật lý được chỉ định.
4. Cập nhật giá trị xuất: Hàm ledcWrite(pin, duty) nhận giá trị, dịch trái 4 bit (chuẩn $Q17.4$ của phần cứng), ghi vào thanh ghi LEDC_HSCHx_DUTY_REG và tự động kích hoạt bit chốt DUTY_START.

API	Luồng xử lý chính	Tác động lên thanh ghi
ledcAttach(pin, freq, resolution)	Tìm Timer/Channel rảnh; tính hệ số chia; cấu hình Timer; kết nối tín hiệu PWM tới GPIO Matrix	Ghi LEDC_HSTIMERx_CONF_REG; sau đó sử dụng esp_rom_gpio_connect_out_signal() để đưa tín hiệu ra chân vật lý
ledcWrite(pin, duty)	Dịch giá trị duty sang định dạng phần cứng; ghi vào thanh ghi duty; kích hoạt chốt giá trị	LEDC_HSCHx_DUTY_REG; giá trị duty được dịch trái 4 bit

2.3. Cơ chế đồng bộ Mutex trên Bus I2C

Sự giao tiếp song song của Task hiển thị LCD và Task đọc cảm biến AHT20 đòi hỏi cơ chế bảo vệ Bus nghiêm ngặt để chống xung đột luồng. Hệ thống I2C trên ESP32 cần sử dụng hai lớp Mutex độc lập để chống xung đột dữ liệu:

1. Mutex nội bộ của Driver: Hệ thống tự động khóa và nhả để bảo vệ tính toàn vẹn của từng giao dịch truyền nhận vật lý riêng lẻ, đảm bảo một gói tin không bị can thiệp giữa chừng.
2. Mutex tầng ứng dụng: Do người lập trình tạo thêm để khóa bảo vệ toàn bộ một chu trình logic kéo dài. Cụ thể, Mutex này khóa chặn quá trình đọc AHT20 (từ lúc gửi lệnh, chờ 80ms đến lúc đọc kết quả) nhằm ngăn Task LCD chen ngang chiếm bus I2C trong 80ms chờ đợi, tránh làm loạn địa chỉ phần cứng.

Khi ứng dụng chạy trên FreeRTOS, việc truy cập bus I2C cần được tuần tự hóa để tránh nhiều Task đồng thời thao tác trên cùng một tài nguyên. Thư viện thực hiện điều này bằng Mutex phần mềm kết hợp với cơ chế chờ sự kiện phần cứng.

Giai đoạn	Thao tác	Ý nghĩa
Khởi tạo	Tạo FreeRTOS Mutex bằng xSemaphoreCreateMutex()	Bảo vệ tài nguyên bus dùng chung
Bắt đầu truyền	xSemaphoreTake() khóa Mutex	Task hiện tại được quyền sử dụng bus
Chờ phần cứng	CPU chuyển Task sang trạng thái Block trong khi phần cứng I2C thực thi	Giảm việc bận chờ (busy waiting), nhường CPU cho Task khác
Kết thúc truyền	Ngắt phần cứng kích hoạt xSemaphoreGive()	Nhả Mutex để Task khác có thể truy cập bus

2.4. Giao thức truyền nhận của Cảm biến và Màn hình

- Chu trình đo và ghép dữ liệu của thư viện AHTX0

Bước	Thao tác	Kết quả
1	Gửi lệnh kích hoạt đo 0xAC qua I2C	Khởi động phép đo
2	Chờ khoảng 80 ms	Đợi cảm biến hoàn tất chuyển đổi
3	Đọc 7 byte từ FIFO I2C	Nhận trạng thái và dữ liệu nhiệt độ / độ ẩm
4	Ghép bit dữ liệu	Tách và tính giá trị nhiệt độ, độ ẩm từ dữ liệu thô

- LiquidCrytal_I2C điều khiển LCD thông qua PCF8574: LCD 16x2 sử dụng PCF8574 làm IC mở rộng I/O giao tiếp I2C, trong khi LCD vẫn được điều khiển theo giao thức song song 4-bit. Vì vậy, mỗi byte dữ liệu/command cần được tách thành phần cao và phần thấp, sau đó tạo xung EN qua PCF8574 để chốt từng nửa byte.

Thành phần	Chuỗi thao tác	Vai trò
4 bit cao	Gửi 4 bit cao + EN = 1; sau đó gửi lại với EN = 0	Tạo xung chốt cho nửa byte cao
4 bit thấp	Gửi 4 bit thấp + EN = 1; sau đó gửi lại với EN = 0	Tạo xung chốt cho nửa byte thấp
Tổng thể	Mỗi byte được truyền qua nhiều lần ghi I2C tới PCF8574	Mô phỏng giao tiếp LCD 4-bit thông qua bus I2C